

module-3-practical-ml-house-prices

July 13, 2026

```
[1]: """
Danielle Gauthier MSDS 422 Practical ML Module 3
House Prices - Advanced Regression Techniques

"""

import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.model_selection import KFold, cross_val_score
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
import matplotlib.pyplot as plt

# Load dataset with headers
train_data = '/kaggle/input/house-prices-advanced-regression-techniques/train.
↳csv'
dataframe = pd.read_csv(train_data)

# Drop ID column (not useful for prediction)
if 'Id' in dataframe.columns:
    dataframe = dataframe.drop('Id', axis=1)

# Separate input features (X) and target variable (y)
X = dataframe.drop('SalePrice', axis=1)
y = dataframe['SalePrice']

# Identify categorical and numerical columns
cat_cols = X.select_dtypes(include=['object']).columns
num_cols = X.select_dtypes(exclude=['object']).columns

# Impute missing values
num_imputer = SimpleImputer(strategy='median')
cat_imputer = SimpleImputer(strategy='most_frequent')

X[num_cols] = num_imputer.fit_transform(X[num_cols])
```

```

X[cat_cols] = cat_imputer.fit_transform(X[cat_cols])

# Convert categorical columns to numerical (one-hot encoding)
X = pd.get_dummies(X, drop_first=True)

print("Data shape after encoding:", X.shape, y.shape)

# Calculate percentage of missing values per column
missing_pct = dataframe.isnull().mean() * 100
missing_pct = missing_pct[missing_pct > 0].sort_values(ascending=False)

# Plot the missing data distribution
plt.figure(figsize=(12, 6))
sns.barplot(x=missing_pct.values, y=missing_pct.index, palette="viridis")
plt.xlabel("Percentage of Missing Values (%)")
plt.title("Missing Data Profile per Feature")
plt.show()

# Create 10-fold cross-validation procedure
cv = KFold(n_splits=10, shuffle=True, random_state=1)

# Create the model
model = RandomForestRegressor(random_state=1, n_jobs=-1)

# Evaluate model using cross-validation
scores = cross_val_score(model, X, y, scoring='neg_mean_absolute_error', cv=cv,
    ↪n_jobs=-1)
scores = np.absolute(scores)
print('Mean MAE: %.3f (%.3f)' % (np.mean(scores), np.std(scores)))

# Fit model on the full dataset to analyze feature importance
model.fit(X, y)

# Get feature importances
importances = model.feature_importances_
feature_names = X.columns

# Sort features by importance
importance_df = pd.DataFrame({'Feature': feature_names, 'Importance':
    ↪importances})
importance_df = importance_df.sort_values(by='Importance', ascending=False)

# Display top 20 most important features
print("\nTop 20 most important features for predicting house prices:")
print(importance_df.head(20))

```

```

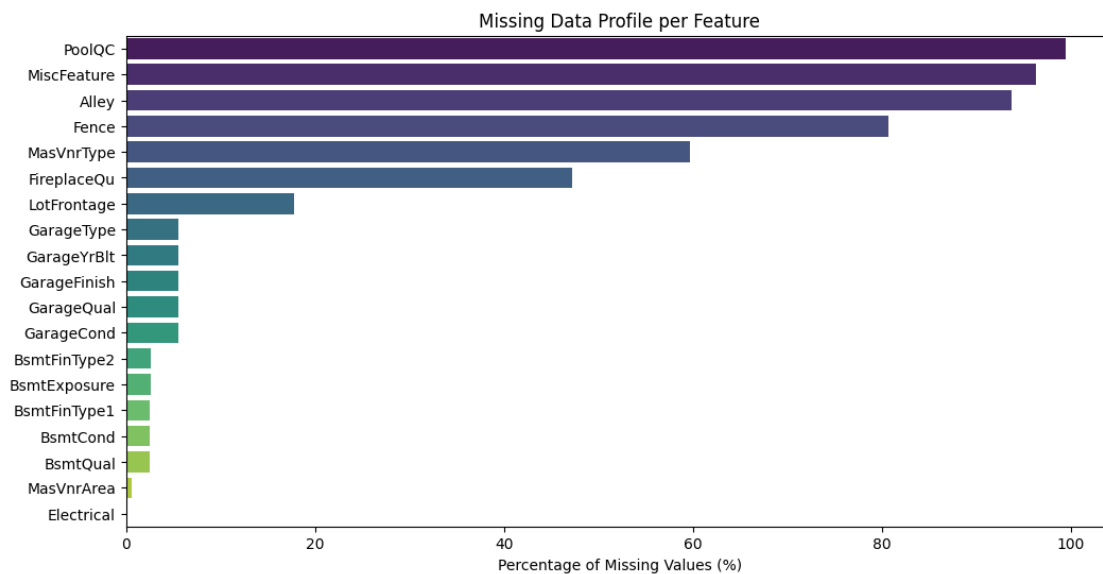
# Plot top 20 features
plt.figure(figsize=(10, 6))
plt.barh(importance_df['Feature'].head(20)[::-1], importance_df['Importance'].
    ↳head(20)[::-1])
plt.xlabel('Feature Importance')
plt.ylabel('Feature')
plt.title('Top 20 Most Important Features for House Price Prediction')
plt.tight_layout()
plt.show()

# Select your top numerical features or top 15 features correlated with
↳SalePrice
top_num_cols = dataframe.select_dtypes(exclude=['object']).corr()['SalePrice'].
    ↳abs().nlargest(15).index
corr_matrix = dataframe[top_num_cols].corr()

plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Correlation Matrix of Top Predictors with SalePrice")
plt.show()

```

Data shape after encoding: (1460, 244) (1460,)

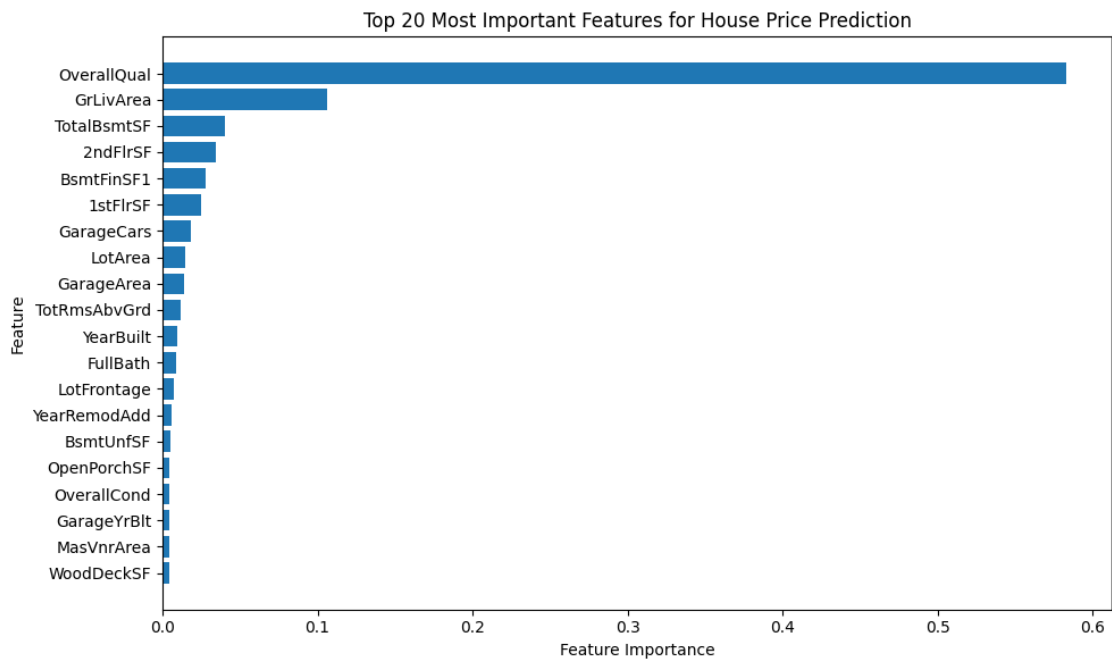


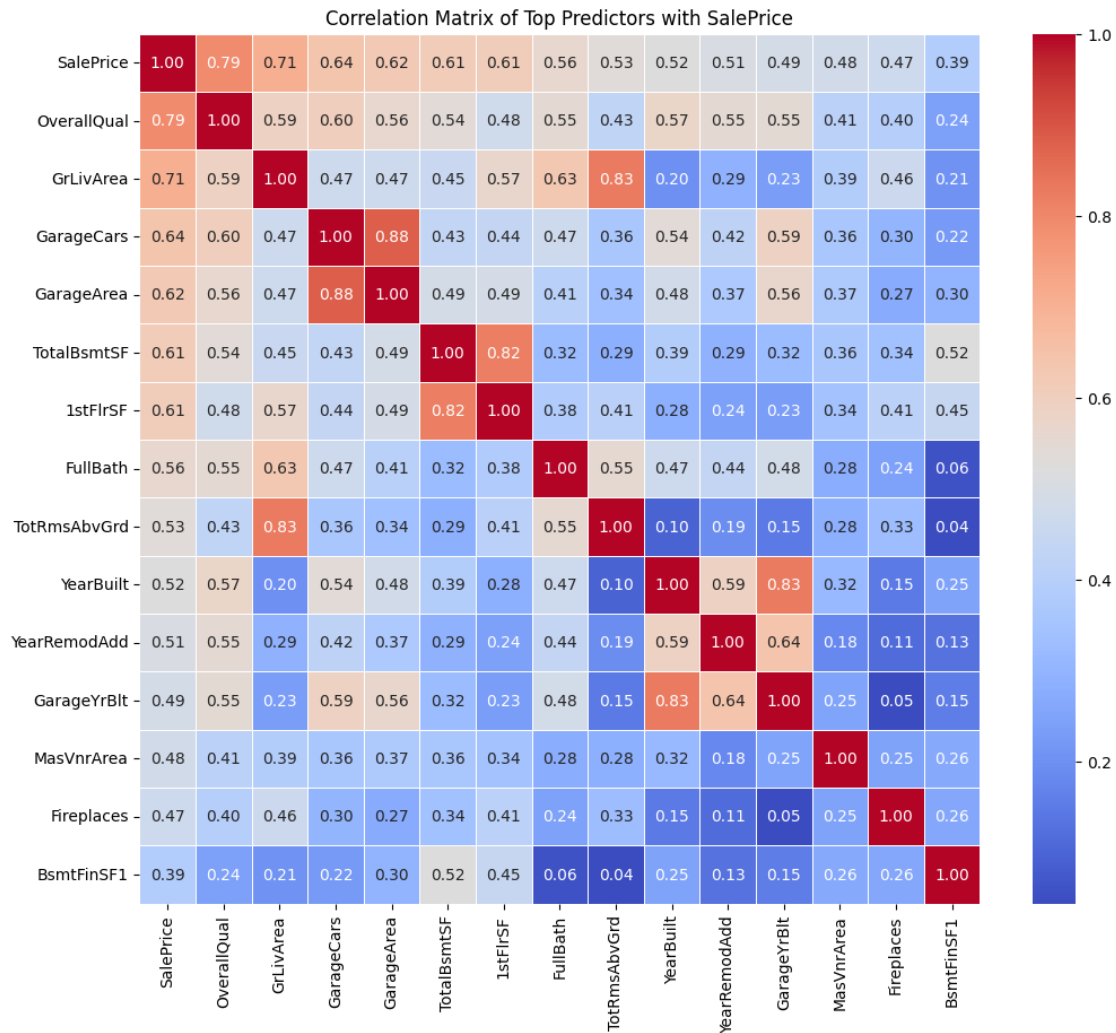
Mean MAE: 17390.392 (1689.894)

Top 20 most important features for predicting house prices:

	Feature	Importance
3	OverallQual	0.583162

15	GrLivArea	0.106460
11	TotalBsmtSF	0.040118
13	2ndFlrSF	0.034640
8	BsmtFinSF1	0.028133
12	1stFlrSF	0.025172
25	GarageCars	0.018078
2	LotArea	0.014840
26	GarageArea	0.013578
22	TotRmsAbvGrd	0.011639
5	YearBuilt	0.009423
18	FullBath	0.008751
1	LotFrontage	0.007290
6	YearRemodAdd	0.005542
10	BsmtUnfSF	0.005053
28	OpenPorchSF	0.004464
4	OverallCond	0.004328
24	GarageYrBlt	0.004320
7	MasVnrArea	0.004177
27	WoodDeckSF	0.004156





```
[2]: import seaborn as sns

# EDA
dataframe.info()

print("\n--- Descriptive Statistics ---")
print(dataframe.describe())

# Distribution of SalePrice
sns.histplot(dataframe['SalePrice'], bins=10, kde=False, color="green")
plt.title("Distribution of SalePrice")
plt.xlabel("SalePrice")
plt.ylabel("Count")
plt.show()
```

```

sns.boxplot(x=dataframe["SalePrice"])
plt.show()

import numpy as np

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Original Target Variable
sns.histplot(dataframe['SalePrice'], bins=30, kde=True, color="green",
             ↪ax=axes[0])
axes[0].set_title("Original SalePrice Distribution")

# Log-Transformed Target Variable
sns.histplot(np.log1p(dataframe['SalePrice']), bins=30, kde=True, color="blue",
             ↪ax=axes[1])
axes[1].set_title("Log-Transformed Log(SalePrice + 1) Distribution")

plt.tight_layout()
plt.show()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Data columns (total 80 columns):
#   Column                Non-Null Count  Dtype
---  -
0   MSSubClass             1460 non-null  int64
1   MSZoning               1460 non-null  object
2   LotFrontage            1201 non-null  float64
3   LotArea               1460 non-null  int64
4   Street                1460 non-null  object
5   Alley                 91 non-null    object
6   LotShape              1460 non-null  object
7   LandContour           1460 non-null  object
8   Utilities             1460 non-null  object
9   LotConfig             1460 non-null  object
10  LandSlope             1460 non-null  object
11  Neighborhood           1460 non-null  object
12  Condition1            1460 non-null  object
13  Condition2            1460 non-null  object
14  BldgType              1460 non-null  object
15  HouseStyle            1460 non-null  object
16  OverallQual           1460 non-null  int64
17  OverallCond           1460 non-null  int64
18  YearBuilt             1460 non-null  int64
19  YearRemodAdd          1460 non-null  int64
20  RoofStyle             1460 non-null  object
21  RoofMatl              1460 non-null  object

```

22	Exterior1st	1460	non-null	object
23	Exterior2nd	1460	non-null	object
24	MasVnrType	588	non-null	object
25	MasVnrArea	1452	non-null	float64
26	ExterQual	1460	non-null	object
27	ExterCond	1460	non-null	object
28	Foundation	1460	non-null	object
29	BsmtQual	1423	non-null	object
30	BsmtCond	1423	non-null	object
31	BsmtExposure	1422	non-null	object
32	BsmtFinType1	1423	non-null	object
33	BsmtFinSF1	1460	non-null	int64
34	BsmtFinType2	1422	non-null	object
35	BsmtFinSF2	1460	non-null	int64
36	BsmtUnfSF	1460	non-null	int64
37	TotalBsmtSF	1460	non-null	int64
38	Heating	1460	non-null	object
39	HeatingQC	1460	non-null	object
40	CentralAir	1460	non-null	object
41	Electrical	1459	non-null	object
42	1stFlrSF	1460	non-null	int64
43	2ndFlrSF	1460	non-null	int64
44	LowQualFinSF	1460	non-null	int64
45	GrLivArea	1460	non-null	int64
46	BsmtFullBath	1460	non-null	int64
47	BsmtHalfBath	1460	non-null	int64
48	FullBath	1460	non-null	int64
49	HalfBath	1460	non-null	int64
50	BedroomAbvGr	1460	non-null	int64
51	KitchenAbvGr	1460	non-null	int64
52	KitchenQual	1460	non-null	object
53	TotRmsAbvGrd	1460	non-null	int64
54	Functional	1460	non-null	object
55	Fireplaces	1460	non-null	int64
56	FireplaceQu	770	non-null	object
57	GarageType	1379	non-null	object
58	GarageYrBlt	1379	non-null	float64
59	GarageFinish	1379	non-null	object
60	GarageCars	1460	non-null	int64
61	GarageArea	1460	non-null	int64
62	GarageQual	1379	non-null	object
63	GarageCond	1379	non-null	object
64	PavedDrive	1460	non-null	object
65	WoodDeckSF	1460	non-null	int64
66	OpenPorchSF	1460	non-null	int64
67	EnclosedPorch	1460	non-null	int64
68	3SsnPorch	1460	non-null	int64
69	ScreenPorch	1460	non-null	int64

```

70 PoolArea      1460 non-null  int64
71 PoolQC        7 non-null    object
72 Fence         281 non-null  object
73 MiscFeature   54 non-null    object
74 MiscVal       1460 non-null  int64
75 MoSold        1460 non-null  int64
76 YrSold        1460 non-null  int64
77 SaleType      1460 non-null  object
78 SaleCondition 1460 non-null  object
79 SalePrice     1460 non-null  int64

```

dtypes: float64(3), int64(34), object(43)

memory usage: 912.6+ KB

--- Descriptive Statistics ---

	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	\
count	1460.000000	1201.000000	1460.000000	1460.000000	1460.000000	
mean	56.897260	70.049958	10516.828082	6.099315	5.575342	
std	42.300571	24.284752	9981.264932	1.382997	1.112799	
min	20.000000	21.000000	1300.000000	1.000000	1.000000	
25%	20.000000	59.000000	7553.500000	5.000000	5.000000	
50%	50.000000	69.000000	9478.500000	6.000000	5.000000	
75%	70.000000	80.000000	11601.500000	7.000000	6.000000	
max	190.000000	313.000000	215245.000000	10.000000	9.000000	

	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	BsmtFinSF2	...	\
count	1460.000000	1460.000000	1452.000000	1460.000000	1460.000000	...	
mean	1971.267808	1984.865753	103.685262	443.639726	46.549315	...	
std	30.202904	20.645407	181.066207	456.098091	161.319273	...	
min	1872.000000	1950.000000	0.000000	0.000000	0.000000	...	
25%	1954.000000	1967.000000	0.000000	0.000000	0.000000	...	
50%	1973.000000	1994.000000	0.000000	383.500000	0.000000	...	
75%	2000.000000	2004.000000	166.000000	712.250000	0.000000	...	
max	2010.000000	2010.000000	1600.000000	5644.000000	1474.000000	...	

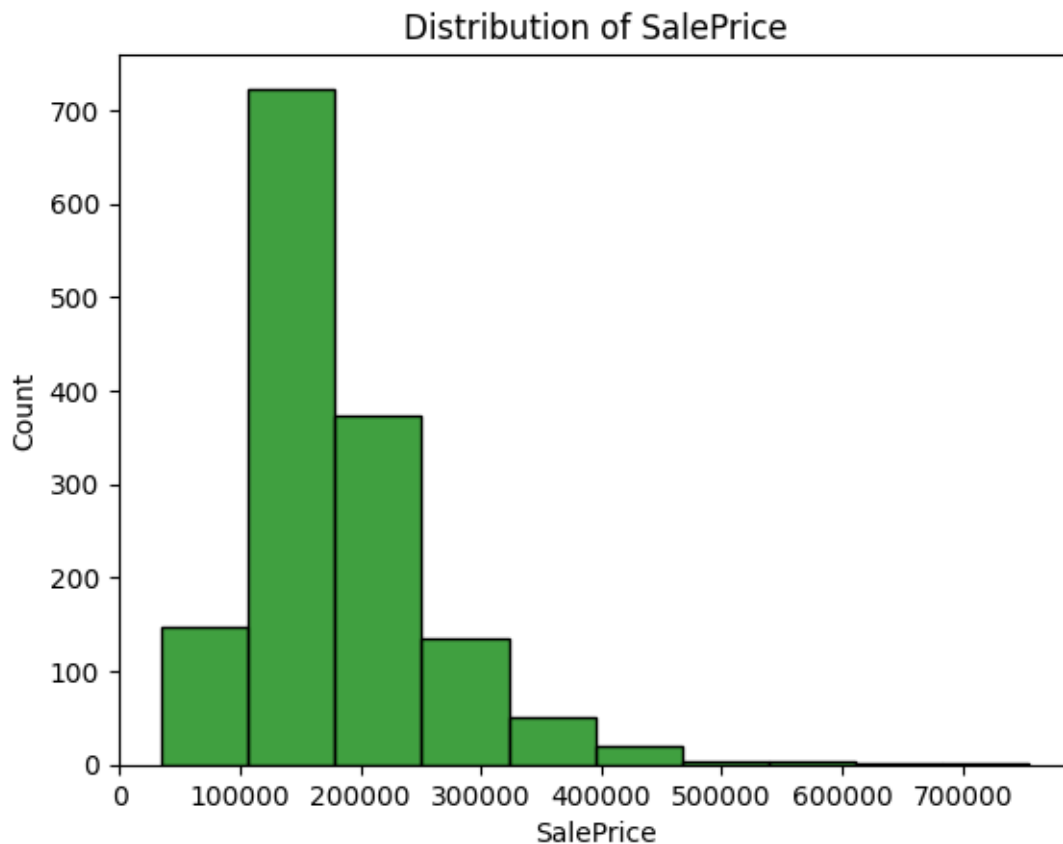
	WoodDeckSF	OpenPorchSF	EnclosedPorch	3SsnPorch	ScreenPorch	\
count	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	
mean	94.244521	46.660274	21.954110	3.409589	15.060959	
std	125.338794	66.256028	61.119149	29.317331	55.757415	
min	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	0.000000	0.000000	0.000000	
50%	0.000000	25.000000	0.000000	0.000000	0.000000	
75%	168.000000	68.000000	0.000000	0.000000	0.000000	
max	857.000000	547.000000	552.000000	508.000000	480.000000	

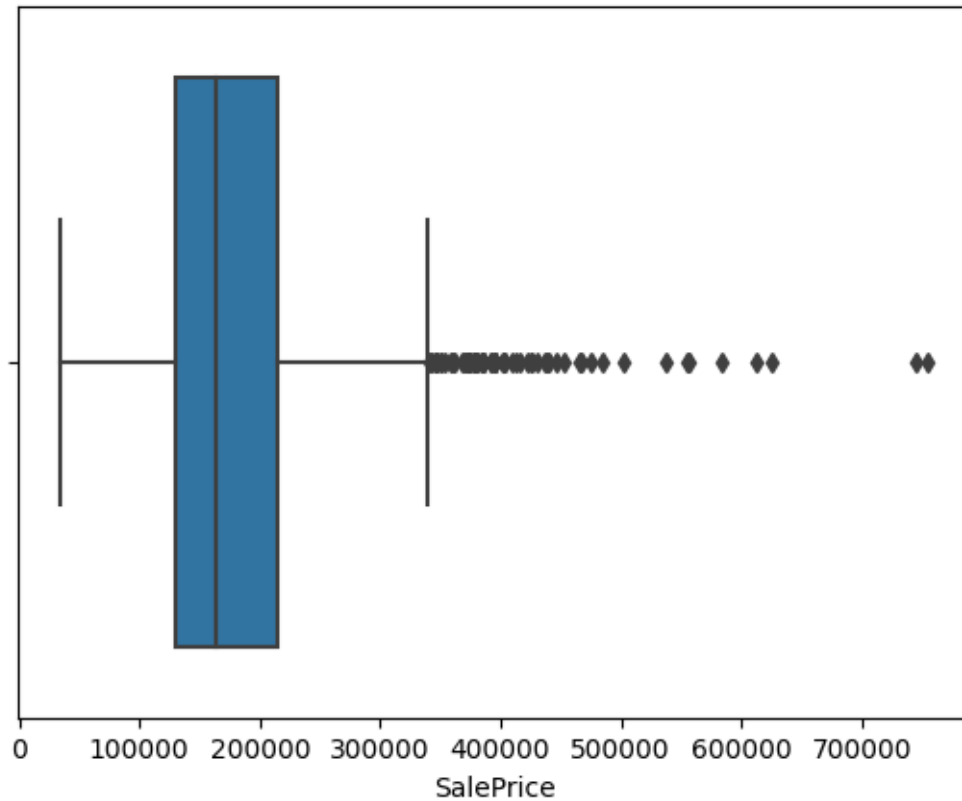
	PoolArea	MiscVal	MoSold	YrSold	SalePrice
count	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000
mean	2.758904	43.489041	6.321918	2007.815753	180921.195890
std	40.177307	496.123024	2.703626	1.328095	79442.502883

min	0.000000	0.000000	1.000000	2006.000000	34900.000000
25%	0.000000	0.000000	5.000000	2007.000000	129975.000000
50%	0.000000	0.000000	6.000000	2008.000000	163000.000000
75%	0.000000	0.000000	8.000000	2009.000000	214000.000000
max	738.000000	15500.000000	12.000000	2010.000000	755000.000000

[8 rows x 37 columns]

```
/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning:
use_inf_as_na option is deprecated and will be removed in a future version.
Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```

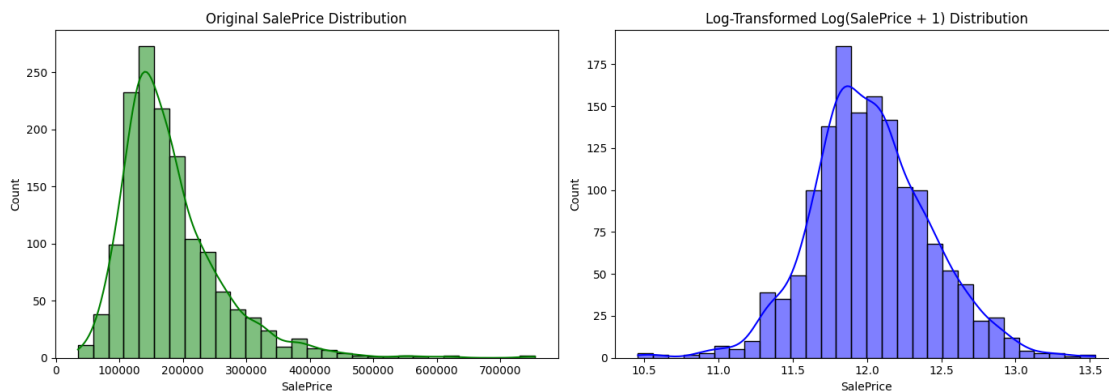




```

/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning:
use_inf_as_na option is deprecated and will be removed in a future version.
Convert inf values to NaN before operating instead.
    with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.11/dist-packages/seaborn/_oldcore.py:1119: FutureWarning:
use_inf_as_na option is deprecated and will be removed in a future version.
Convert inf values to NaN before operating instead.
    with pd.option_context('mode.use_inf_as_na', True):

```



```
[3]: from statsmodels.formula.api import ols

# Select outcome variable
y = dataframe['SalePrice']

# Select features
X = dataframe.drop('SalePrice', axis=1)

# Combine back for OLS
df_ols = dataframe.copy()

# Ensure OverallQual is numeric
print(df_ols['OverallQual'].dtype)

# Write the linear regression formula
ols_formula = "SalePrice ~ OverallQual"

# Implement OLS regression
model = ols(formula=ols_formula, data=df_ols).fit()

# Display summary
print(model.summary())

# Plot regression line
sns.regplot(x="OverallQual", y="SalePrice", data=df_ols, line_kws={"color": "red", "↪"red"})
plt.title("OLS Regression: SalePrice vs OverallQual")
plt.xlabel("Overall Quality")
plt.ylabel("Sale Price")
plt.show()
```

int64

OLS Regression Results

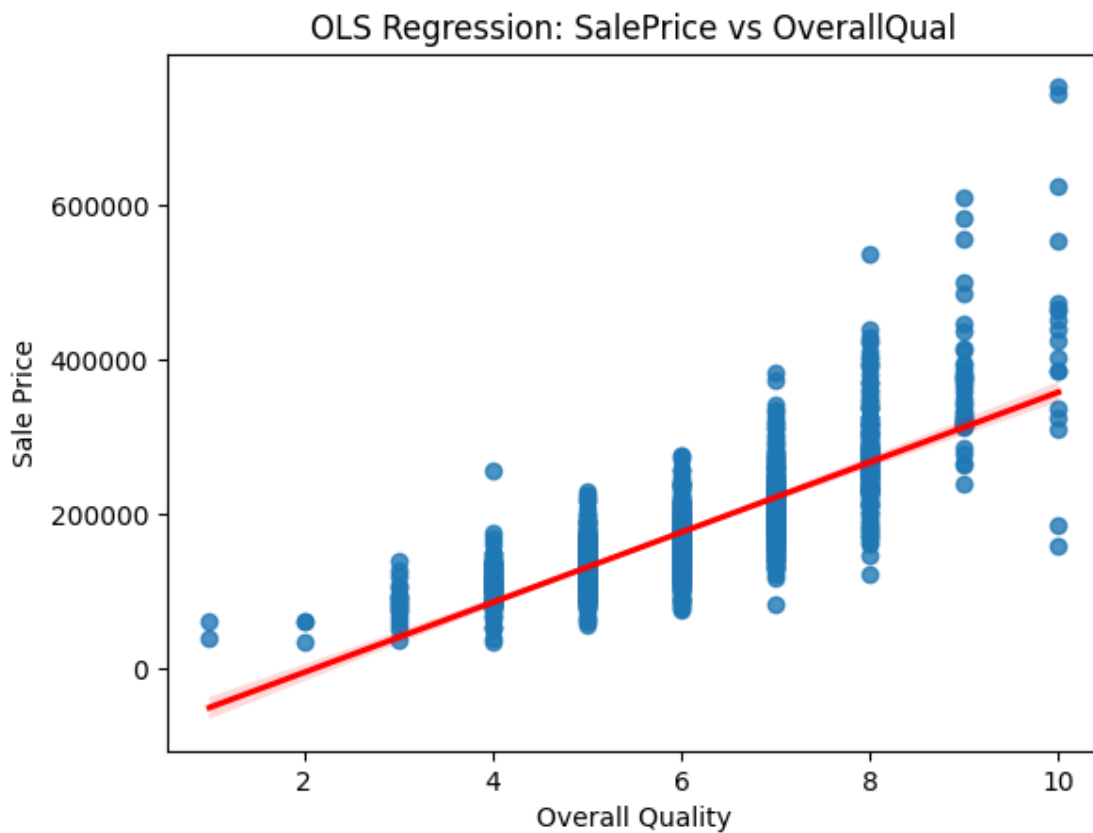
```
=====
Dep. Variable:          SalePrice    R-squared:                0.626
Model:                  OLS         Adj. R-squared:           0.625
Method:                 Least Squares   F-statistic:            2437.
Date:                   Mon, 13 Jul 2026   Prob (F-statistic):     2.19e-313
Time:                   12:35:52         Log-Likelihood:         -17827.
No. Observations:      1460            AIC:                   3.566e+04
Df Residuals:          1458            BIC:                   3.567e+04
Df Model:               1
Covariance Type:        nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]

Intercept	-9.621e+04	5756.407	-16.713	0.000	-1.07e+05	-8.49e+04
OverallQual	4.544e+04	920.430	49.364	0.000	4.36e+04	4.72e+04
=====						
Omnibus:		591.594	Durbin-Watson:			1.990
Prob(Omnibus):		0.000	Jarque-Bera (JB):			5872.098
Skew:		1.607	Prob(JB):			0.00
Kurtosis:		12.284	Cond. No.			29.0
=====						

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.



```
[4]: import statsmodels.api as sm

# Select outcome variable
y = dataframe['SalePrice']

# Select features
X = dataframe.drop('SalePrice', axis=1)
```

```

# Combine back for OLS
df_ols = dataframe.copy()

# Ensure predictors are numeric
print(df_ols[['OverallQual', 'GrLivArea', 'TotalBsmtSF']].dtypes)

# Write the multiple linear regression formula
ols_formula = "SalePrice ~ OverallQual + GrLivArea + TotalBsmtSF"

# Implement OLS regression
multiple_model = ols(formula=ols_formula, data=df_ols).fit()

# Display summary
print(multiple_model.summary())

# Partial regression plots for each predictor
fig = plt.figure(figsize=(10, 6))
sm.graphics.plot_partregress_grid(multiple_model, fig=fig)
plt.tight_layout()
plt.show()

```

```

OverallQual    int64
GrLivArea      int64
TotalBsmtSF    int64
dtype: object

```

OLS Regression Results

```

=====
Dep. Variable:          SalePrice    R-squared:                0.742
Model:                  OLS         Adj. R-squared:           0.742
Method:                 Least Squares    F-statistic:            1396.
Date:                   Mon, 13 Jul 2026    Prob (F-statistic):      0.00
Time:                   12:36:07           Log-Likelihood:         -17555.
No. Observations:       1460             AIC:                   3.512e+04
Df Residuals:           1456             BIC:                   3.514e+04
Df Model:                3
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-1.038e+05	4794.565	-21.651	0.000	-1.13e+05	-9.44e+04
OverallQual	2.805e+04	1023.743	27.396	0.000	2.6e+04	3.01e+04
GrLivArea	49.4526	2.551	19.388	0.000	44.449	54.456
TotalBsmtSF	36.6150	2.918	12.548	0.000	30.891	42.339

```

=====
Omnibus:                525.531    Durbin-Watson:           1.972
Prob(Omnibus):           0.000    Jarque-Bera (JB):        45766.824
Skew:                    -0.726    Prob(JB):                 0.00

```

Kurtosis: 30.390 Cond. No. 8.89e+03

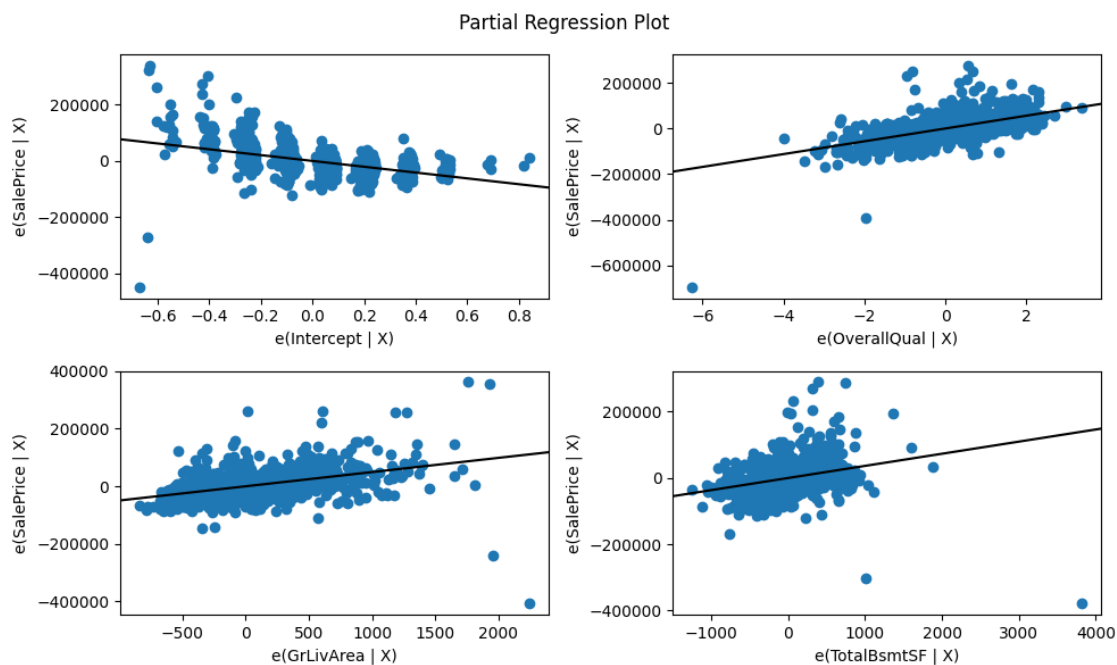
Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 8.89e+03. This might indicate that there are strong multicollinearity or other numerical problems.

/tmp/ipykernel_40/1406404913.py:27: UserWarning: The figure layout has changed to tight

```
plt.tight_layout()
```



```
[10]: import numpy as np
import pandas as pd
from statsmodels.formula.api import ols

# 1. Create the cubic feature alongside your existing quadratic feature
df_ols['GrLivArea_cub'] = df_ols['GrLivArea'] ** 3

# 2. Fit the new cubic model
cubic_model = ols("SalePrice ~ GrLivArea + GrLivArea_sq + GrLivArea_cub",
    ↪data=df_ols).fit()
print(cubic_model.summary())

# 3. Plot the Cubic Regression Line
```

```

plt.figure(figsize=(10, 6))
sns.scatterplot(x='GrLivArea', y='SalePrice', data=df_ols, alpha=0.5,
               label='Actual Data')

# Generate smooth x-values across the range of GrLivArea to draw the cubic curve
x_vals = np.linspace(df_ols['GrLivArea'].min(), df_ols['GrLivArea'].max(), 200)
prediction_df = pd.DataFrame({
    'GrLivArea': x_vals,
    'GrLivArea_sq': x_vals ** 2,
    'GrLivArea_cub': x_vals ** 3
})
y_vals = cubic_model.predict(prediction_df)

plt.plot(x_vals, y_vals, color='red', linewidth=2.5, label='Cubic Fit')
plt.title("Cubic Polynomial Regression: SalePrice vs GrLivArea")
plt.xlabel("Above Ground Living Area (GrLivArea)")
plt.ylabel("Sale Price")
plt.legend()
plt.show()

```

OLS Regression Results

```

=====
Dep. Variable:          SalePrice    R-squared:                0.521
Model:                  OLS         Adj. R-squared:           0.520
Method:                 Least Squares    F-statistic:            527.7
Date:                   Mon, 13 Jul 2026    Prob (F-statistic):      4.72e-232
Time:                   12:41:11          Log-Likelihood:         -18007.
No. Observations:       1460             AIC:                   3.602e+04
Df Residuals:           1456             BIC:                   3.604e+04
Df Model:                3
Covariance Type:        nonrobust
=====
=
              coef      std err          t      P>|t|      [0.025
0.975]
-----
-
Intercept      7.421e+04   1.66e+04     4.478     0.000     4.17e+04
1.07e+05
GrLivArea      -0.0657     25.407    -0.003     0.998    -49.904
49.772
GrLivArea_sq     0.0596      0.012     5.136     0.000      0.037
0.082
GrLivArea_cub  -9.426e-06   1.53e-06    -6.145     0.000    -1.24e-05
-6.42e-06
=====
Omnibus:            308.629    Durbin-Watson:           2.011
Prob(Omnibus):      0.000    Jarque-Bera (JB):        1622.264

```

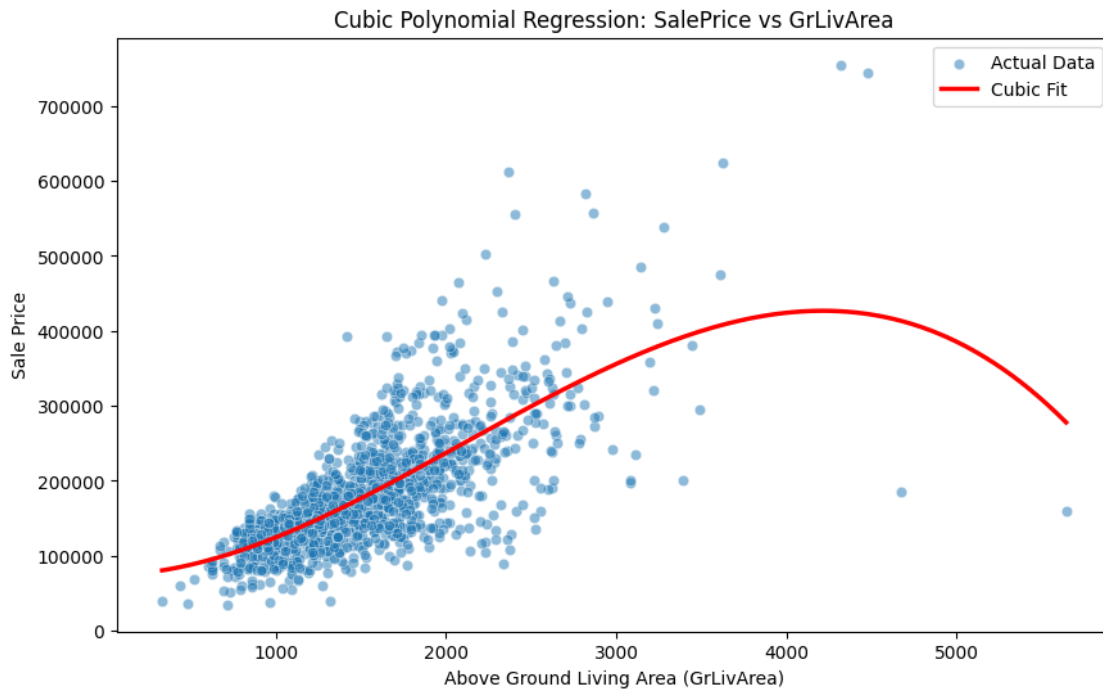
Skew:	0.883	Prob(JB):	0.00
Kurtosis:	7.852	Cond. No.	1.08e+11

=====

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.08e+11. This might indicate that there are strong multicollinearity or other numerical problems.



```
[7]: from statsmodels.stats.anova import anova_lm

# Indicator model
cat_model = ols("SalePrice ~ C(Neighborhood)", data=df_ols).fit()
print(cat_model.summary())

# Evaluate
anova_lm(cat_model)

plt.figure(figsize=(14, 8))
# Sort neighborhoods by median sale price to easily spot variance
neighborhood_order = dataframe.groupby('Neighborhood')['SalePrice'].median().
    ↪sort_values().index
```



```

sns.boxplot(x='Neighborhood', y='SalePrice', data=dataframe,
            order=neighborhood_order)
plt.xticks(rotation=45, ha='right')
plt.title("SalePrice Distribution Across Neighborhoods (Sorted by Median)")
plt.tight_layout()
plt.show()

```

OLS Regression Results

```

=====
Dep. Variable:          SalePrice      R-squared:                0.546
Model:                  OLS           Adj. R-squared:           0.538
Method:                 Least Squares  F-statistic:              71.78
Date:                   Mon, 13 Jul 2026  Prob (F-statistic):      1.56e-225
Time:                   12:36:48       Log-Likelihood:           -17968.
No. Observations:      1460          AIC:                     3.599e+04
Df Residuals:          1435          BIC:                     3.612e+04
Df Model:               24
Covariance Type:       nonrobust
=====

```

```

=====

```

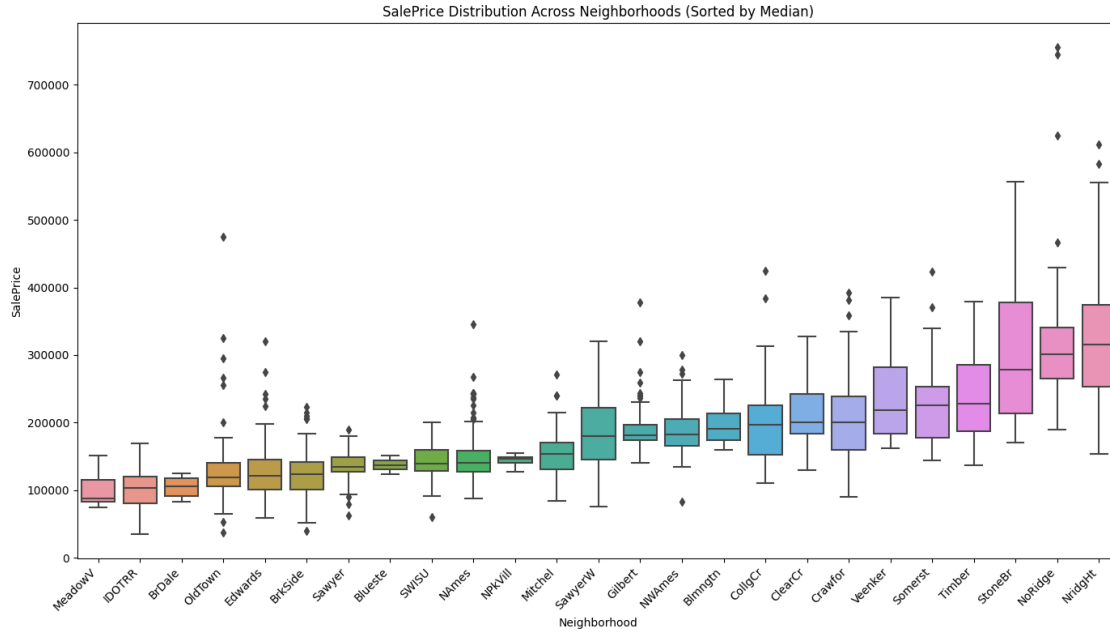
		coef	std err	t	P> t
[0.025	0.975]				

Intercept		1.949e+05	1.31e+04	14.879	0.000
1.69e+05	2.21e+05				
C(Neighborhood) [T.Blueste]		-5.737e+04	4.04e+04	-1.421	0.155
-1.37e+05	2.18e+04				
C(Neighborhood) [T.BrDale]		-9.038e+04	1.88e+04	-4.805	0.000
-1.27e+05	-5.35e+04				
C(Neighborhood) [T.BrkSide]		-7.004e+04	1.49e+04	-4.703	0.000
-9.93e+04	-4.08e+04				
C(Neighborhood) [T.ClearCr]		1.769e+04	1.66e+04	1.066	0.287
-1.49e+04	5.03e+04				
C(Neighborhood) [T.CollgCr]		3094.8910	1.38e+04	0.224	0.823
-2.4e+04	3.02e+04				
C(Neighborhood) [T.Crawfor]		1.575e+04	1.51e+04	1.042	0.298
-1.39e+04	4.54e+04				
C(Neighborhood) [T.Edwards]		-6.665e+04	1.42e+04	-4.705	0.000
-9.44e+04	-3.89e+04				
C(Neighborhood) [T.Gilbert]		-2016.3760	1.44e+04	-0.140	0.889
-3.03e+04	2.63e+04				
C(Neighborhood) [T.IDOTRR]		-9.475e+04	1.58e+04	-5.988	0.000
-1.26e+05	-6.37e+04				
C(Neighborhood) [T.MeadowV]		-9.629e+04	1.85e+04	-5.199	0.000
-1.33e+05	-6e+04				
C(Neighborhood) [T.Mitchel]		-3.86e+04	1.52e+04	-2.540	0.011
-6.84e+04	-8784.735				

C(Neighborhood) [T.NAmes]	-4.902e+04	1.36e+04	-3.609	0.000
-7.57e+04	-2.24e+04			
C(Neighborhood) [T.NPkVill]	-5.218e+04	2.23e+04	-2.344	0.019
-9.58e+04	-8510.657			
C(Neighborhood) [T.NWAmes]	-5820.8139	1.45e+04	-0.400	0.689
-3.43e+04	2.27e+04			
C(Neighborhood) [T.NoRidge]	1.404e+05	1.56e+04	9.015	0.000
1.1e+05	1.71e+05			
C(Neighborhood) [T.NridgHt]	1.214e+05	1.45e+04	8.390	0.000
9.3e+04	1.5e+05			
C(Neighborhood) [T.OldTown]	-6.665e+04	1.4e+04	-4.744	0.000
-9.42e+04	-3.91e+04			
C(Neighborhood) [T.SWISU]	-5.228e+04	1.7e+04	-3.080	0.002
-8.56e+04	-1.9e+04			
C(Neighborhood) [T.Sawyer]	-5.808e+04	1.45e+04	-3.999	0.000
-8.66e+04	-2.96e+04			
C(Neighborhood) [T.SawyerW]	-8315.0857	1.49e+04	-0.559	0.576
-3.75e+04	2.08e+04			
C(Neighborhood) [T.Somerst]	3.051e+04	1.43e+04	2.129	0.033
2393.494	5.86e+04			
C(Neighborhood) [T.StoneBr]	1.156e+05	1.7e+04	6.812	0.000
8.23e+04	1.49e+05			
C(Neighborhood) [T.Timber]	4.738e+04	1.58e+04	3.007	0.003
1.65e+04	7.83e+04			
C(Neighborhood) [T.Veenker]	4.39e+04	2.09e+04	2.101	0.036
2913.679	8.49e+04			
=====				
Omnibus:	618.883	Durbin-Watson:	1.956	
Prob(Omnibus):	0.000	Jarque-Bera (JB):	5526.438	
Skew:	1.737	Prob(JB):	0.00	
Kurtosis:	11.875	Cond. No.	48.8	
=====				

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.



```
[8]: # Dichotomous
df_ols['HasGarage'] = (df_ols['GarageCars'] > 0).astype(int)

dichotomous_model = ols("SalePrice ~ HasGarage", data=df_ols).fit()
print(dichotomous_model.summary())

# Evaluate
sns.boxplot(x='HasGarage', y='SalePrice', data=df_ols)
plt.title("Effect of Having a Garage on SalePrice")
plt.show()
```

OLS Regression Results

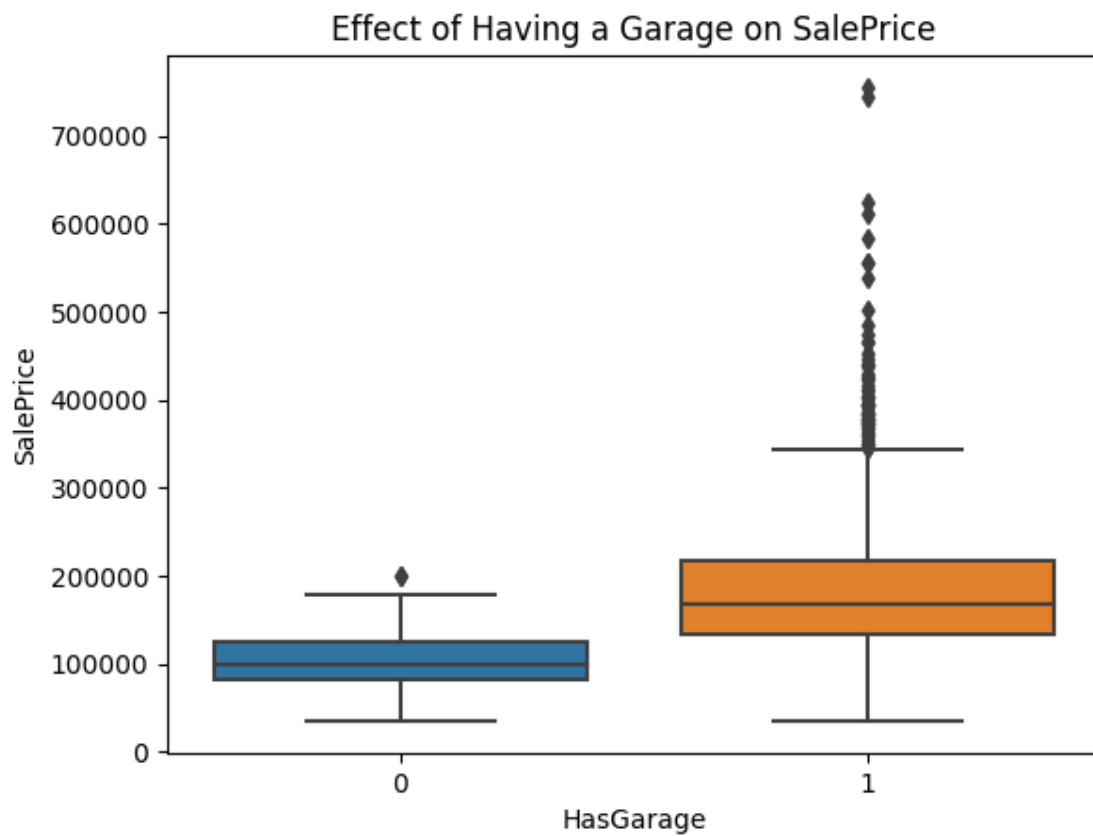
=====						
Dep. Variable:	SalePrice		R-squared:	0.056		
Model:	OLS		Adj. R-squared:	0.055		
Method:	Least Squares		F-statistic:	86.64		
Date:	Mon, 13 Jul 2026		Prob (F-statistic):	4.63e-20		
Time:	12:37:49		Log-Likelihood:	-18502.		
No. Observations:	1460		AIC:	3.701e+04		
Df Residuals:	1458		BIC:	3.702e+04		
Df Model:	1					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

Intercept	1.033e+05	8578.765	12.043	0.000	8.65e+04	1.2e+05
HasGarage	8.216e+04	8827.120	9.308	0.000	6.48e+04	9.95e+04

```
=====
Omnibus:                643.100    Durbin-Watson:                2.019
Prob(Omnibus):           0.000    Jarque-Bera (JB):           3986.594
Skew:                    1.962    Prob(JB):                   0.00
Kurtosis:                10.081    Cond. No.                   8.38
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.



```
[9]: # Define piecewise variable
df_ols['GrLivArea_piece'] = np.where(df_ols['GrLivArea'] > 2000,
    ↪ df_ols['GrLivArea'] - 2000, 0)

# Fit piecewise regression
piecewise_model = ols("SalePrice ~ GrLivArea + GrLivArea_piece", data=df_ols).
    ↪ fit()
print(piecewise_model.summary())
```

```
# Evaluate
sns.scatterplot(x='GrLivArea', y='SalePrice', data=df_ols)
x_vals = np.linspace(df_ols['GrLivArea'].min(), df_ols['GrLivArea'].max(), 100)
plt.plot(x_vals, piecewise_model.predict(pd.DataFrame({'GrLivArea': x_vals,
↳ 'GrLivArea_piece': np.maximum(x_vals - 2000, 0)})), color='red')
plt.title("Piecewise Linear Relationship: SalePrice vs GrLivArea")
plt.show()
```

OLS Regression Results

```
=====
Dep. Variable:          SalePrice    R-squared:                0.504
Model:                  OLS         Adj. R-squared:            0.504
Method:                 Least Squares   F-statistic:             741.2
Date:                  Mon, 13 Jul 2026   Prob (F-statistic):       9.08e-223
Time:                  12:37:54         Log-Likelihood:          -18032.
No. Observations:      1460            AIC:                    3.607e+04
Df Residuals:          1457            BIC:                    3.609e+04
Df Model:               2
Covariance Type:       nonrobust
=====
```

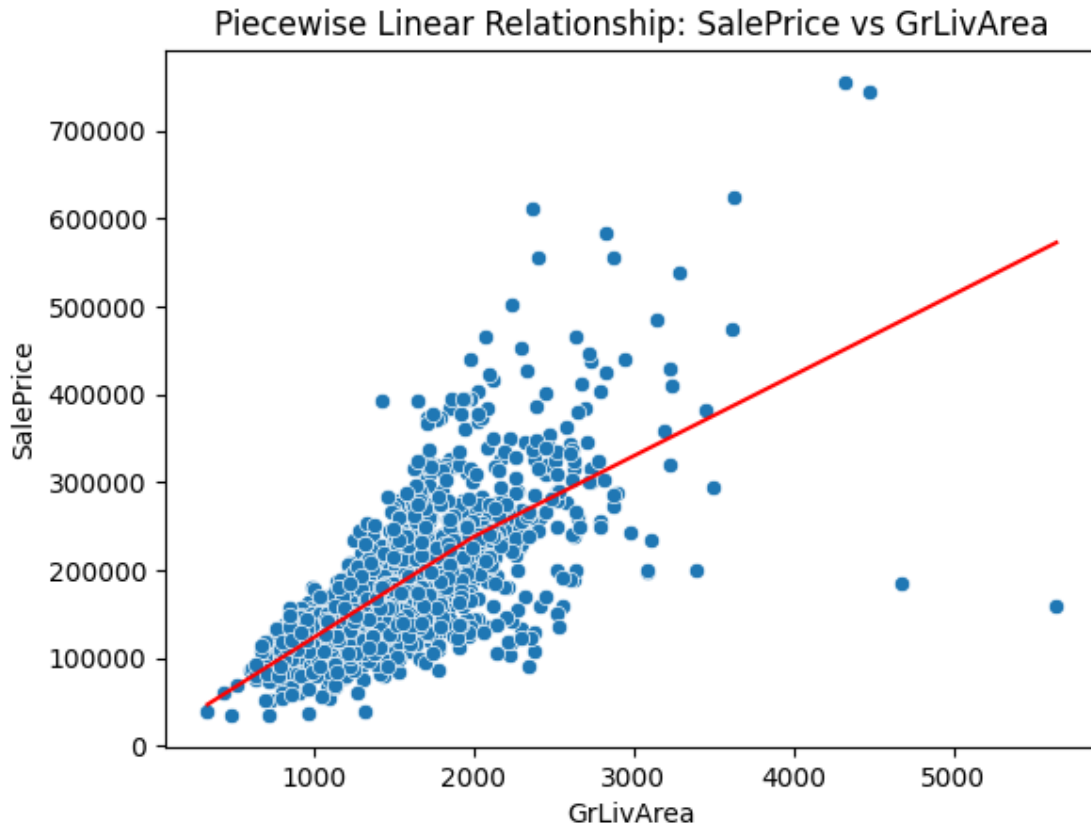
```
===
               coef      std err          t      P>|t|      [0.025
0.975]
```

```
-----
---
Intercept      8569.7806    5976.113      1.434      0.152    -3152.924
2.03e+04
GrLivArea      114.6874      4.093     28.023      0.000     106.659
122.715
GrLivArea_piece -22.8249      9.048     -2.523      0.012    -40.573
-5.077
```

```
=====
Omnibus:                281.298    Durbin-Watson:           2.021
Prob(Omnibus):           0.000    Jarque-Bera (JB):        2614.191
Skew:                    0.615    Prob(JB):                 0.00
Kurtosis:                9.439    Cond. No.                 6.56e+03
=====
```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 6.56e+03. This might indicate that there are strong multicollinearity or other numerical problems.



```
[26]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf

# A. Standardize features first so scale differences don't distort penalties
df_scaled = df_ols.copy()
features_to_scale = ['OverallQual', 'GrLivArea', 'TotalBsmtSF', 'GarageCars']

for col in features_to_scale:
    df_scaled[col] = (df_scaled[col] - df_scaled[col].mean()) / df_scaled[col].
    ↪std()

# Define your feature formula
formula = "SalePrice ~ OverallQual + GrLivArea + TotalBsmtSF + GarageCars"

# B. Fit the base OLS model to use as a starting point
size_qual_model = smf.ols(formula, data=df_ols)
```

```

# 1. RIDGE REGRESSION (L1_wt = 0)
# alpha controls the penalty strength. L1_wt=0 forces pure L2 penalty.
s_q_ridge_model = size_qual_model.fit_regularized(alpha=0.1, L1_wt=0.0)

# 2. LASSO REGRESSION (L1_wt = 1)
# L1_wt=1 forces pure L1 penalty, driving less important coefficients to
↳ exactly 0.
s_q_lasso_model = size_qual_model.fit_regularized(alpha=0.1, L1_wt=1.0)

# 3. ELASTIC NET REGRESSION (0 < L1_wt < 1)
# Combines both penalties. Here we give an equal 50/50 blend.
s_q_elastic_net_model = size_qual_model.fit_regularized(alpha=0.1, L1_wt=0.5)

print("Ridge Model:", s_q_ridge_model.params)

print("Lasso Model:", s_q_lasso_model.params)

print("Elastic Net:", s_q_elastic_net_model.params)

# C. Create a grid of alpha values to sweep across (from tiny to large
↳ penalties)
alphas = np.logspace(-1, 5, 100)

# Data structures to store coefficient trajectories
ridge_coefs = []
lasso_coefs = []

# Loop over alphas and extract parameters
for a in alphas:
    ridge_coefs.append(s_q_ridge_model.params)
    lasso_coefs.append(s_q_lasso_model.params)

# Convert to DataFrames for plotting (ignoring Intercept to focus on feature
↳ weights)
df_ridge = pd.DataFrame(ridge_coefs, index=alphas).drop(columns=['Intercept'],
↳ errors='ignore')
df_lasso = pd.DataFrame(lasso_coefs, index=alphas).drop(columns=['Intercept'],
↳ errors='ignore')

# 4. Plot the Paths side-by-side
fig, axes = plt.subplots(1, 2, figsize=(15, 6), sharey=True)

# Ridge Path Plot
for column in df_ridge.columns:
    axes[0].plot(df_ridge.index, df_ridge[column], label=column, linewidth=2)
axes[0].set_xscale('log')
axes[0].set_title('Ridge Regression Path (L2 Penalty)')

```

```

axes[0].set_xlabel('Regularization Strength (Alpha)')
axes[0].set_ylabel('Coefficient Weight')
axes[0].grid(True, linestyle="--", alpha=0.5)

# Lasso Path Plot
for column in df_lasso.columns:
    axes[1].plot(df_lasso.index, df_lasso[column], label=column, linewidth=2)
axes[1].set_xscale('log')
axes[1].set_title('Lasso Regression Path (L1 Penalty)')
axes[1].set_xlabel('Regularization Strength (Alpha)')
axes[1].grid(True, linestyle="--", alpha=0.5)

# Add a single legend for clarity
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

```

Ridge Model: [-31287.48703005 13294.93531484 44.94256839 34.42208764
16829.42687207]

Lasso Model: Intercept -106737.167782

OverallQual 25673.890146

GrLivArea 46.231091

TotalBsmtSF 31.487004

GarageCars 15776.423594

dtype: float64

Elastic Net: Intercept -57361.547326

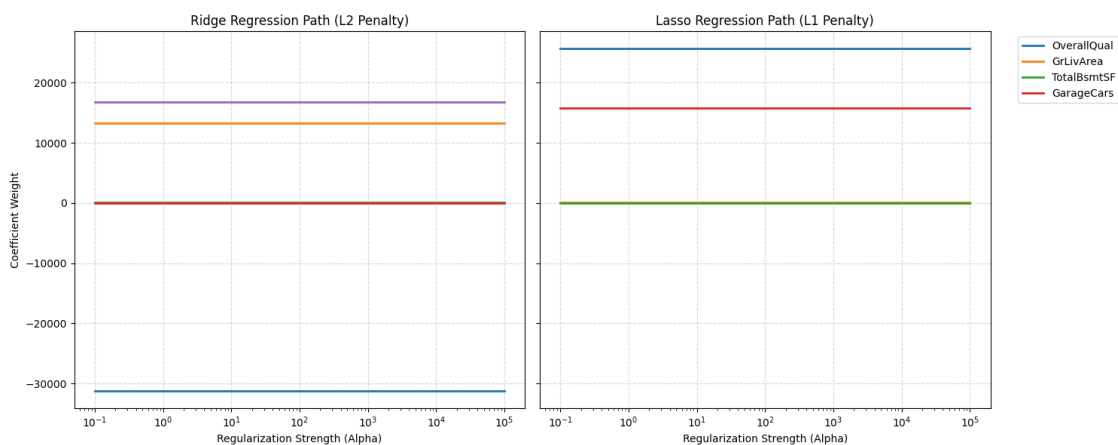
OverallQual 17604.924796

GrLivArea 46.137831

TotalBsmtSF 33.231191

GarageCars 15962.141313

dtype: float64




```

[27]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf

# A. Standardize features first so scale differences don't distort penalties
df_scaled = df_ols.copy()
features_to_scale = ['YearBuilt', 'YearRemodAdd', 'OverallCond']

for col in features_to_scale:
    df_scaled[col] = (df_scaled[col] - df_scaled[col].mean()) / df_scaled[col].
    ↪std()

# Define your feature formula
formula = "SalePrice ~ YearBuilt + YearRemodAdd + OverallCond"

# B. Fit the base OLS model to use as a starting age_rene_qual_model = smf.
    ↪ols(formula, data=df_ols)
age_ren_model = smf.ols(formula, data=df_ols)

# 1. RIDGE REGRESSION (L1_wt = 0)
# alpha controls the penalty strength. L1_wt=0 forces pure L2 penalty.
a_r_ridge_model = age_ren_model.fit_regularized(alpha=0.1, L1_wt=0.0)

# 2. LASSO REGRESSION (L1_wt = 1)
# L1_wt=1 forces pure L1 penalty, driving less important coefficients to
    ↪exactly 0.
a_r_lasso_model = age_ren_model.fit_regularized(alpha=0.1, L1_wt=1.0)

# 3. ELASTIC NET REGRESSION (0 < L1_wt < 1)
# Combines both penalties. Here we give an equal 50/50 blend.
a_r_elastic_net_model = age_ren_model.fit_regularized(alpha=0.1, L1_wt=0.5)

print("Ridge Model:", a_r_ridge_model.params)

print("Lasso Model:", a_r_lasso_model.params)

print("Elastic Net:", a_r_elastic_net_model.params)

# C. Create a grid of alpha values to sweep across (from tiny to large
    ↪penalties)
alphas = np.logspace(-1, 5, 100)

# Data structures to store coefficient trajectories
ridge_coefs = []

```

```

lasso_coefs = []

# Loop over alphas and extract parameters
for a in alphas:
    ridge_coefs.append(a_r_ridge_model.params)
    lasso_coefs.append(a_r_lasso_model.params)

# Convert to DataFrames for plotting (ignoring Intercept to focus on feature
↪weights)
df_ridge = pd.DataFrame(ridge_coefs, index=alphas).drop(columns=['Intercept'],
↪errors='ignore')
df_lasso = pd.DataFrame(lasso_coefs, index=alphas).drop(columns=['Intercept'],
↪errors='ignore')

# 4. Plot the Paths side-by-side
fig, axes = plt.subplots(1, 2, figsize=(15, 6), sharey=True)

# Ridge Path Plot
for column in df_ridge.columns:
    axes[0].plot(df_ridge.index, df_ridge[column], label=column, linewidth=2)
axes[0].set_xscale('log')
axes[0].set_title('Ridge Regression Path (L2 Penalty)')
axes[0].set_xlabel('Regularization Strength (Alpha)')
axes[0].set_ylabel('Coefficient Weight')
axes[0].grid(True, linestyle="--", alpha=0.5)

# Lasso Path Plot
for column in df_lasso.columns:
    axes[1].plot(df_lasso.index, df_lasso[column], label=column, linewidth=2)
axes[1].set_xscale('log')
axes[1].set_title('Lasso Regression Path (L1 Penalty)')
axes[1].set_xlabel('Regularization Strength (Alpha)')
axes[1].grid(True, linestyle="--", alpha=0.5)

# Add a single legend for clarity
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

```

Ridge Model: [-4186.67708112 748.25090142 -652.47124734 1002.89089012]

Lasso Model: Intercept 195208.405480

YearBuilt 15.390534

YearRemodAdd -4.670517

OverallCond -6305.193158

dtype: float64

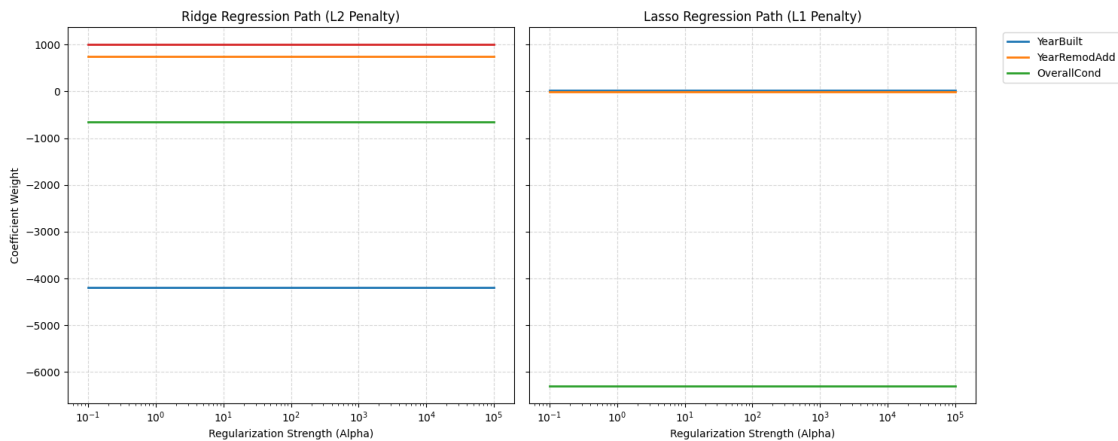
Elastic Net: Intercept 15487.975684

YearBuilt 103.843938

```

YearRemodAdd      -4.315859
OverallCond       -5459.915582
dtype: float64

```



```

[66]: import numpy as np
import pandas as pd
from sklearn.model_selection import KFold
import statsmodels.formula.api as smf

# 1. Map your specific workspace variables
elastic_net_models = {
    "s_q_elastic_net_model": s_q_elastic_net_model,
    "a_r_elastic_net_model": a_r_elastic_net_model
}

# 2. Set up the hyperparameter grid to search
alphas = [0.001, 0.01, 0.1, 1.0, 10.0]
l1_wts = [0.1, 0.3, 0.5, 0.7, 0.9]

# 3. K-Fold configuration
cv = KFold(n_splits=5, shuffle=True, random_state=1)

# Loop through your exact workspace models
for name, model_obj in elastic_net_models.items():
    print(f"\n===== Tuning: {name} =====")

    # Safely pull the formula string directly from your model instance
    if hasattr(model_obj, 'model'):
        formula = model_obj.model.formula
    else:
        formula = model_obj.formula

```

```

best_mae = float('inf')
best_params = {}

# Grid search loop
for alpha in alphas:
    for l1_wt in l1_wts:
        mae_scores = []

        for train_idx, test_idx in cv.split(df_clean):
            train_fold = df_clean.iloc[train_idx]
            test_fold = df_clean.iloc[test_idx]

            try:
                # Fit regularized elastic net on the train fold
                fold_model = smf.ols(formula, data=train_fold).
↪fit_regularized(
                    alpha=alpha,
                    L1_wt=l1_wt
                )

                # Generate predictions
                log_preds = fold_model.predict(test_fold)
                actual_dollars = test_fold['SalePrice']

                # Inverse log if your formulas use Log_SalePrice
                if "Log_SalePrice" in formula:
                    dollar_preds = np.expm1(log_preds)
                else:
                    dollar_preds = log_preds

                mae = np.mean(np.abs(actual_dollars - dollar_preds))
                mae_scores.append(mae)

            except Exception:
                continue

        if mae_scores:
            mean_cv_mae = np.mean(mae_scores)
            if mean_cv_mae < best_mae:
                best_mae = mean_cv_mae
                best_params = {'alpha': alpha, 'L1_wt': l1_wt}

print(f" Best Parameters Found for {name}:")
print(f" -> Alpha: {best_params.get('alpha')}")
print(f" -> L1 Weight (Ratio): {best_params.get('L1_wt')}")
print(f" -> Minimum Out-of-Sample MAE: ${best_mae:,.2f}")

```

===== Tuning: s_q_elastic_net_model =====

Best Parameters Found for s_q_elastic_net_model:

-> Alpha: 0.1
-> L1 Weight (Ratio): 0.7
-> Minimum Out-of-Sample MAE: \$24,944.33

===== Tuning: a_r_elastic_net_model =====

Best Parameters Found for a_r_elastic_net_model:

-> Alpha: 1.0
-> L1 Weight (Ratio): 0.9
-> Minimum Out-of-Sample MAE: \$55,559.96

```
[28]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.formula.api as smf

# 1. Start with a fresh copy of your data
df_scaled = df_ols.copy()

# 2. ONLY scale the numerical variables for Model 3
features_to_scale = ['LotArea']

for col in features_to_scale:
    df_scaled[col] = (df_scaled[col] - df_scaled[col].mean()) / df_scaled[col].
    ↪std()

# 3. Define the formula (Patsy automatically handles the string 'Neighborhood' ↪
    ↪column)
formula = "SalePrice ~ Neighborhood + LotArea"
loca_model = smf.ols(formula, data=df_scaled)

# 4. Create a grid of alpha values to sweep across
alphas = np.logspace(-1, 4, 100)

ridge_coefs = []
lasso_coefs = []

for a in alphas:
    # fit_regularized treats categorical dummy columns fairly once numericals ↪
    ↪are scaled
    l_ridge_fit = loca_model.fit_regularized(alpha=a, L1_wt=0.0)
    l_lasso_fit = loca_model.fit_regularized(alpha=a, L1_wt=1.0)

    ridge_coefs.append(l_ridge_fit.params)
    lasso_coefs.append(l_lasso_fit.params)
```

```

# Convert to DataFrames (excluding intercept)
df_ridge = pd.DataFrame(ridge_coefs, index=alphas).drop(columns=['Intercept'],
↳errors='ignore')
df_lasso = pd.DataFrame(lasso_coefs, index=alphas).drop(columns=['Intercept'],
↳errors='ignore')

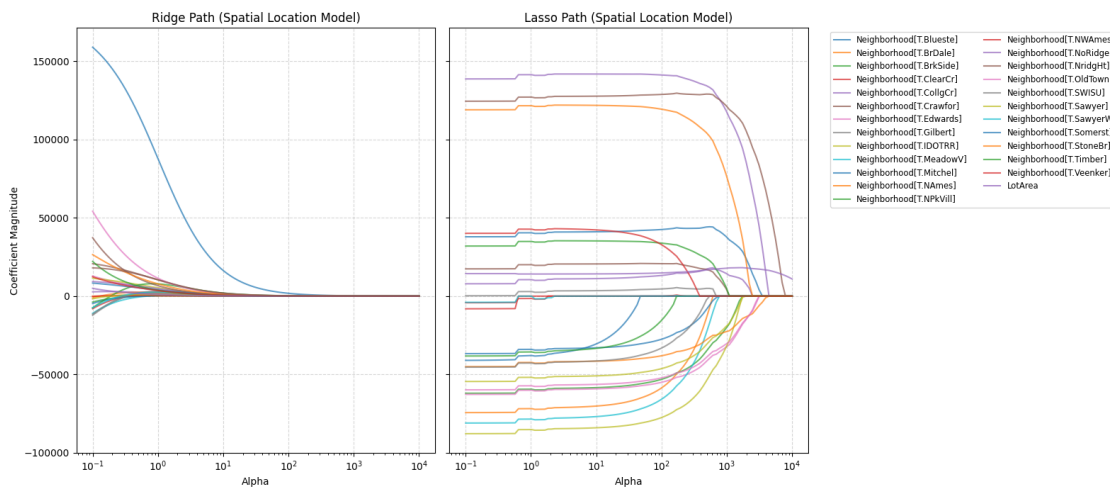
# 5. Plot the Trajectories
fig, axes = plt.subplots(1, 2, figsize=(16, 7), sharey=True)

# Ridge Path (Notice how the neighborhood impacts shrink but stay alive)
for column in df_ridge.columns:
    axes[0].plot(df_ridge.index, df_ridge[column], label=column, alpha=0.7)
axes[0].set_xscale('log')
axes[0].set_title('Ridge Path (Spatial Location Model)')
axes[0].set_xlabel('Alpha')
axes[0].set_ylabel('Coefficient Magnitude')
axes[0].grid(True, linestyle="--", alpha=0.5)

# Lasso Path (Watch which neighborhoods get driven completely to zero first)
for column in df_lasso.columns:
    axes[1].plot(df_lasso.index, df_lasso[column], label=column, alpha=0.7)
axes[1].set_xscale('log')
axes[1].set_title('Lasso Path (Spatial Location Model)')
axes[1].set_xlabel('Alpha')
axes[1].grid(True, linestyle="--", alpha=0.5)

# Optional: Place legend outside since there are many neighborhoods
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', ncol=2, fontsize='small')
plt.tight_layout()
plt.show()

```



```

[39]: import numpy as np
import pandas as pd
from sklearn.model_selection import KFold
import statsmodels.formula.api as smf

# 1. Ensure your models dictionary is populated correctly
models = {
    "Linear (GrLivArea)":      smf.ols("SalePrice ~ GrLivArea", data=df_ols).
    ↪fit(),
    "Polynomial (GrLivArea^3)": cubic_model, # Ensure your cubic model_
    ↪variable matches here
    "Piecewise (GrLivArea)":   piecewise_model,
    "Indicator (Neighborhood)": cat_model,
    "Multiple (3 predictors)":  multiple_model,
    "Dichotomous (HasGarage)":  dichotomous_model,
    "Size and Quality":         size_qual_model,
    "Age and Renovation":       age_ren_model,
    "Location":                 loca_model
}

# 2. Setup your Cross-Validation framework
cv = KFold(n_splits=10, shuffle=True, random_state=1)
cv_results = {}

print(f"{'Model':<28} | {'Mean CV MAE':<12} | {'In-Sample AIC':<12}")
print("-" * 60)

# 3. Calculate metrics for every model dynamically
for name, model in models.items():
    try:
        # Extract formula and data frame whether it is already fitted or not
        if hasattr(model, 'model'):
            formula = model.model.formula
            data = model.model.data.frame
        else:
            formula = model.formula
            data = model.data.frame

        # Safely pull AIC only if it's a standard OLS model, otherwise flag it_
        ↪N/A
        aic_val = f"{model.aic:.1f}" if hasattr(model, 'aic') else "N/A_
        ↪(Regularized)"

        mae_scores = []

        # Manual K-Fold loop to accommodate statsmodels' regularized framework
        for train_idx, test_idx in cv.split(data):

```

```

train_fold = data.iloc[train_idx]
test_fold = data.iloc[test_idx]

# Instantiate a fresh version of the current model on the training_
↪slice

fold_base = smf.ols(formula, data=train_fold)

# Mirror original regularization hyperparameters
if "Ridge" in name:
    fold_model = fold_base.fit_regularized(alpha=0.1, L1_wt=0.0)
elif "Lasso" in name:
    fold_model = fold_base.fit_regularized(alpha=0.1, L1_wt=1.0)
elif "Elastic" in name:
    fold_model = fold_base.fit_regularized(alpha=0.1, L1_wt=0.5)
else:
    fold_model = fold_base.fit()

predictions = fold_model.predict(test_fold)
actuals = test_fold['SalePrice']
mae_scores.append(np.mean(np.abs(actuals - predictions)))

mean_mae = np.mean(mae_scores)
cv_results[name] = (mean_mae, aic_val)

except Exception as e:
    print(f" Skipping '{name}' due to error: {e}")

# 4. Sort models by predictive performance (Lower MAE = Better Prediction Power)
sorted_results = sorted(cv_results.items(), key=lambda x: x[1][0])

# 5. Output the final leaderboard safely
if sorted_results:
    for name, (mae, aic) in sorted_results:
        print(f"{name:<28} | {mae:<12.2f} | {aic:<12}")

    best_predictive_model = sorted_results[0][0]
    print(f"\nBest model for predictive power (Lowest MAE):_
↪{best_predictive_model}")
else:
    print("\n Error: No models were successfully evaluated. Check the error_
↪logs above.")

```

Model	Mean CV MAE	In-Sample AIC
Size and Quality	25379.56	N/A (Regularized)
Multiple (3 predictors)	26143.16	35117.7
Location	35304.46	N/A (Regularized)

Indicator (Neighborhood)	36580.50	35986.5
Linear (GrLivArea)	37797.81	36073.8
Polynomial (GrLivArea ³)	38185.37	36021.6
Piecewise (GrLivArea)	38212.32	36069.4
Age and Renovation	44934.58	N/A (Regularized)
Dichotomous (HasGarage)	55473.82	37007.8

Best model for predictive power (Lowest MAE): Size and Quality

Indicator Model best fit

```
[41]: import matplotlib.pyplot as plt
import seaborn as sns

# 1. If size_qual_model is un-fitted, call .fit() to get the results object
if not hasattr(size_qual_model, 'predict_proba') and hasattr(size_qual_model, 'fit'):
    fitted_size_qual = size_qual_model.fit()
else:
    fitted_size_qual = size_qual_model

# 2. Generate predictions from the fitted results object
predictions = fitted_size_qual.predict(df_ols)
residuals = df_ols['SalePrice'] - predictions

# 3. Create the residual plot
plt.figure(figsize=(10, 6))
sns.scatterplot(x=predictions, y=residuals, alpha=0.5, color='purple')
plt.axhline(y=0, color='red', linestyle='--', linewidth=2)
plt.title("Residual Plot: Size and Quality Model")
plt.xlabel("Predicted SalePrice")
plt.ylabel("Residuals (Actual - Predicted)")
plt.grid(True, linestyle="--", alpha=0.5)
plt.show()
```



```
[43]: import statsmodels.formula.api as smf

# Create consolidated space features
df_ols['TotalRealEstateSF'] = df_ols['GrLivArea'] + df_ols['TotalBsmtSF']

# Create an interaction term: Quality * Living Area
# (Houses with huge square footage should scale exponentially higher if quality
# is top-tier)
df_ols['Qual_x_Area'] = df_ols['OverallQual'] * df_ols['GrLivArea']

# Build an updated multiple regression model with your interaction structures
enhanced_formula = "SalePrice ~ Qual_x_Area + TotalRealEstateSF + GarageCars + YearBuilt"
enhanced_model = smf.ols(enhanced_formula, data=df_ols).fit()

print(enhanced_model.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          SalePrice    R-squared:                0.768
Model:                  OLS         Adj. R-squared:           0.767
Method:                 Least Squares   F-statistic:             1204.
Date:                  Mon, 13 Jul 2026   Prob (F-statistic):       0.00
Time:                  14:04:53         Log-Likelihood:          -17477.
No. Observations:      1460            AIC:                    3.496e+04
```

Df Residuals: 1455 BIC: 3.499e+04
Df Model: 4
Covariance Type: nonrobust

	coef	std err	t	P> t	[0.025
0.975]					

Intercept	-9.503e+05	7.66e+04	-12.407	0.000	-1.1e+06
-8e+05					
Qual_x_Area	8.2789	0.433	19.128	0.000	7.430
9.128					
TotalRealEstateSF	14.6018	2.667	5.476	0.000	9.371
19.833					
GarageCars	1.679e+04	1798.180	9.337	0.000	1.33e+04
2.03e+04					
YearBuilt	499.1265	39.578	12.611	0.000	421.490
576.763					
=====					
Omnibus:	938.658	Durbin-Watson:	1.998		
Prob(Omnibus):	0.000	Jarque-Bera (JB):	128136.991		
Skew:	-2.025	Prob(JB):	0.00		
Kurtosis:	48.716	Cond. No.	8.72e+05		
=====					

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 8.72e+05. This might indicate that there are strong multicollinearity or other numerical problems.

```
[49]: import numpy as np
import statsmodels.formula.api as smf

# 1. Fill empty values referencing df_ols to ensure consistency
for col in ['FullBath', 'BedroomAbvGr', 'YrSold', 'YearBuilt', 'SalePrice']:
    if col in df_ols.columns:
        df_ols[col] = df_ols[col].fillna(df_ols[col].median())

# 2. Fix the division-by-zero issue: If BedroomAbvGr is 0, replace it with 0.5
↳ or 1
# to represent the layout safely without producing infinity ('inf')
safe_bedrooms = df_ols['BedroomAbvGr'].replace(0, 1)
df_ols['BathtoBedRatio'] = df_ols['FullBath'] / safe_bedrooms

# 3. Calculate Age at Sale
```

```

df_ols['AgeatSale'] = df_ols['YrSold'] - df_ols['YearBuilt']

# 4. Explicitly drop any row containing a residual NaN or Inf anywhere in these
↳ columns
columns_to_check = ['SalePrice', 'BathtoBedRatio', 'AgeatSale']
df_clean = df_ols.dropna(subset=columns_to_check).copy()

# Replace any accidental remaining inf values with NaN and drop them
df_clean = df_clean.replace([np.inf, -np.inf], np.nan).
↳ dropna(subset=columns_to_check)

# 5. Build and fit your updated model using the clean dataset
feature_formula = "SalePrice ~ BathtoBedRatio + AgeatSale"
feature_model = smf.ols(feature_formula, data=df_clean).fit()

print(feature_model.summary())

```

OLS Regression Results

```

=====
Dep. Variable:          SalePrice    R-squared:                0.291
Model:                  OLS         Adj. R-squared:            0.290
Method:                 Least Squares    F-statistic:            299.4
Date:                  Mon, 13 Jul 2026    Prob (F-statistic):      1.17e-109
Time:                  14:20:24          Log-Likelihood:          -18293.
No. Observations:      1460             AIC:                   3.659e+04
Df Residuals:          1457             BIC:                   3.661e+04
Df Model:               2
Covariance Type:       nonrobust
=====
==

```

	coef	std err	t	P> t	[0.025
Intercept	1.962e+05	6458.478	30.380	0.000	1.84e+05
BathtoBedRatio	4.946e+04	8271.289	5.979	0.000	3.32e+04
AgeatSale	-1195.9918	65.156	-18.356	0.000	-1323.801

```

-----
--
Omnibus:                734.216    Durbin-Watson:           1.978
Prob(Omnibus):           0.000    Jarque-Bera (JB):        6469.875
Skew:                   2.165    Prob(JB):                0.00
Kurtosis:               12.360    Cond. No.:               278.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[58]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf

# 1. Transform the target variable to log space
df_clean['Log_SalePrice'] = np.log1p(df_clean['SalePrice'])

# 2. Refit your model using the log target
log_enhanced_formula = "Log_SalePrice ~ Qual_x_Area + TotalRealEstateSF +
    ↳GarageCars + YearBuilt"
log_enhanced_model = smf.ols(log_enhanced_formula, data=df_clean).fit()

# 3. When checking residuals for a new plot, use the log space values:
log_enhanced_predictions = log_enhanced_model.predict(df_clean)
log_enhanced_residuals = df_clean['Log_SalePrice'] - log_enhanced_predictions
log_enhanced_preds = log_enhanced_predictions
log_enhanced_resids = log_enhanced_residuals

# 2. Generate dollar-space predictions by reversing the log transformation
dollar_enhanced_preds = np.expml(log_enhanced_preds)
dollar_enhanced_resids = df_clean['SalePrice'] - dollar_enhanced_preds

# 3. Plot both side-by-side to view the output
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Plot A: Log-Space Residuals
sns.scatterplot(x=log_enhanced_preds, y=log_enhanced_resids, alpha=0.5,
    ↳color='darkblue', ax=axes[0])
axes[0].axhline(y=0, color='red', linestyle='--', linewidth=2)
axes[0].set_title("Enhanced Model: Log-Space Residuals\n(Checks regression
    ↳assumptions)")
axes[0].set_xlabel("Predicted Log(SalePrice)")
axes[0].set_ylabel("Residuals (Log Units)")
axes[0].grid(True, linestyle="--", alpha=0.5)

# Plot B: Dollar-Space Residuals
sns.scatterplot(x=dollar_enhanced_preds, y=dollar_enhanced_resids, alpha=0.5,
    ↳color='darkorange', ax=axes[1])
axes[1].axhline(y=0, color='red', linestyle='--', linewidth=2)
axes[1].set_title("Enhanced Model: Real Dollar-Space Residuals\n(Shows actual
    ↳predictive errors)")
```

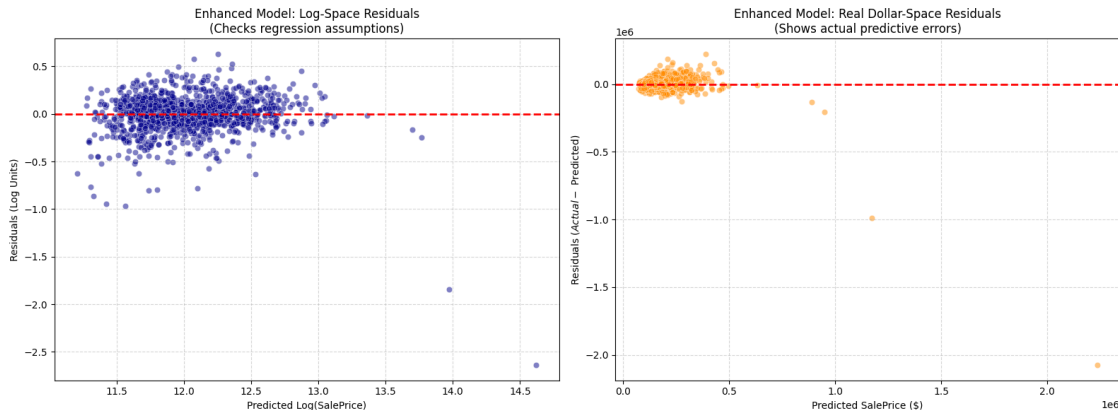
```

axes[1].set_xlabel("Predicted SalePrice ($)")
axes[1].set_ylabel("Residuals ($ Actual - $ Predicted)")
axes[1].grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()

# Print the performance summary
print(f"Enhanced Log Model Adjusted R2: {log_enhanced_model.rsquared_adj:.4f}")

```



Enhanced Log Model Adjusted R²: 0.7790

```

[53]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf

# 1. Transform the target variable to log space
df_clean['Log_SalePrice'] = np.log1p(df_clean['SalePrice'])

# 2. Fit the model using the log target
log_formula = "Log_SalePrice ~ BathtoBedRatio + AgeatSale"
log_model = smf.ols(log_formula, data=df_clean).fit()

# 3. Generate log-space predictions and residuals
log_preds = log_model.predict(df_clean)
log_resids = df_clean['Log_SalePrice'] - log_preds

# 4. Generate dollar-space predictions (using exponential to invert the log)
dollar_preds = np.expml(log_preds)
dollar_resids = df_clean['SalePrice'] - dollar_preds

```

```

# 5. Plot both side-by-side to view the output
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

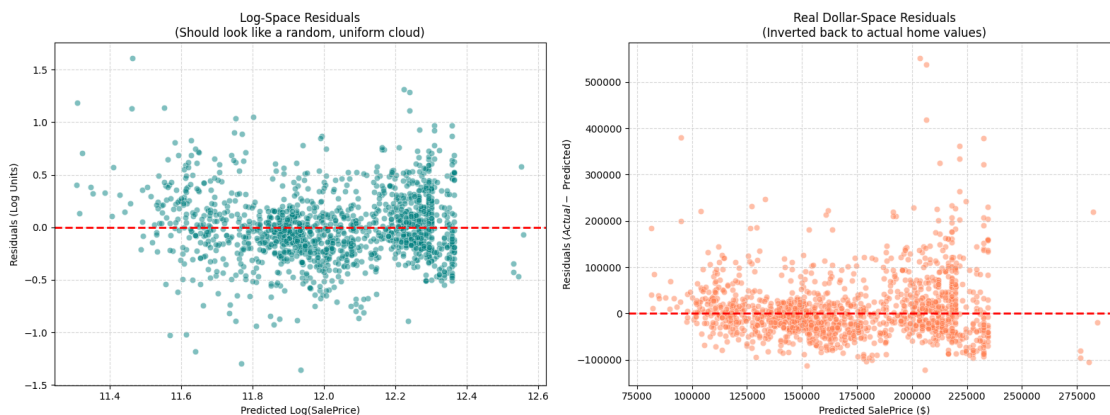
# Plot A: Log-Space Residuals (Checks if regression assumptions are fixed)
sns.scatterplot(x=log_preds, y=log_resids, alpha=0.5, color='teal', ax=axes[0])
axes[0].axhline(y=0, color='red', linestyle='--', linewidth=2)
axes[0].set_title("Log-Space Residuals\n(Should look like a random, uniform cloud)")
axes[0].set_xlabel("Predicted Log(SalePrice)")
axes[0].set_ylabel("Residuals (Log Units)")
axes[0].grid(True, linestyle="--", alpha=0.5)

# Plot B: Dollar-Space Residuals (Shows real-world performance)
sns.scatterplot(x=dollar_preds, y=dollar_resids, alpha=0.5, color='coral', ax=axes[1])
axes[1].axhline(y=0, color='red', linestyle='--', linewidth=2)
axes[1].set_title("Real Dollar-Space Residuals\n(Inverted back to actual home values)")
axes[1].set_xlabel("Predicted SalePrice ($)")
axes[1].set_ylabel("Residuals ($ Actual - $ Predicted)")
axes[1].grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()

# Print the R² performance summary
print(f"Log Model Adjusted R²: {log_model.rsquared_adj:.4f}")

```



Log Model Adjusted R^2 : 0.3546

```

[61]: import numpy as np
import pandas as pd

```

```

from sklearn.model_selection import KFold
import statsmodels.formula.api as smf

# 1. Ensure your models dictionary is populated correctly
models = {
    "Linear (GrLivArea)":      smf.ols("SalePrice ~ GrLivArea", data=df_ols).
    ↪fit(),
    "Polynomial (GrLivArea^3)": cubic_model, # Ensure your cubic model
    ↪variable matches here
    "Piecewise (GrLivArea)":   piecewise_model,
    "Indicator (Neighborhood)": cat_model,
    "Multiple (3 predictors)":  multiple_model,
    "Dichotomous (HasGarage)":  dichotomous_model,
    "Size and Quality":        size_qual_model,
    "Age and Renovation":      age_ren_model,
    "Location":                loca_model,
    "Enhanced Size Quality Model": enhanced_model,
    "Log Enhanced Size Quality Model": log_enhanced_model,
    "Feature Model": feature_model,
    "Log Feature Model": log_model
}

# 2. Setup your Cross-Validation framework
cv = KFold(n_splits=10, shuffle=True, random_state=1)
cv_results = {}

print(f"{'Model':<28} | {'Mean CV MAE':<12} | {'In-Sample AIC':<12}")
print("-" * 60)

# 3. Calculate metrics for every model dynamically
for name, model in models.items():
    try:
        # Extract formula and data frame whether it is already fitted or not
        if hasattr(model, 'model'):
            formula = model.model.formula
            data = model.model.data.frame
        else:
            formula = model.formula
            data = model.data.frame

        # Safely pull AIC only if it's a standard OLS model, otherwise flag it
        ↪N/A
        aic_val = f"{model.aic:.1f}" if hasattr(model, 'aic') else "N/A"
        ↪(Regularized)"

        mae_scores = []

```



```

# Manual K-Fold loop to accommodate statsmodels' regularized framework
for train_idx, test_idx in cv.split(data):
    train_fold = data.iloc[train_idx]
    test_fold = data.iloc[test_idx]

    # Instantiate a fresh version of the current model on the training_
↪ slice
    fold_base = smf.ols(formula, data=train_fold)

    # Mirror original regularization hyperparameters
    if "Ridge" in name:
        fold_model = fold_base.fit_regularized(alpha=0.1, L1_wt=0.0)
    elif "Lasso" in name:
        fold_model = fold_base.fit_regularized(alpha=0.1, L1_wt=1.0)
    elif "Elastic" in name:
        fold_model = fold_base.fit_regularized(alpha=0.1, L1_wt=0.5)
    else:
        fold_model = fold_base.fit()

    predictions = fold_model.predict(test_fold)
    actuals = test_fold['SalePrice']
    mae_scores.append(np.mean(np.abs(actuals - predictions)))

mean_mae = np.mean(mae_scores)
cv_results[name] = (mean_mae, aic_val)

except Exception as e:
    print(f" Skipping '{name}' due to error: {e}")

# 4. Sort models by predictive performance (Lower MAE = Better Prediction Power)
sorted_results = sorted(cv_results.items(), key=lambda x: x[1][0])

# 5. Output the final leaderboard safely
if sorted_results:
    for name, (mae, aic) in sorted_results:
        print(f"{name:<28} | {mae:<12.2f} | {aic:<12}")

    best_predictive_model = sorted_results[0][0]
    print(f"\nBest model for predictive power (Lowest MAE):_
↪ {best_predictive_model}")
else:
    print("\n Error: No models were successfully evaluated. Check the error_
↪ logs above.")

```

Model	Mean CV MAE	In-Sample AIC
Enhanced Size Quality Model	23880.87	34964.7

Size and Quality	25379.56	N/A (Regularized)
Multiple (3 predictors)	26143.16	35117.7
Location	35304.46	N/A (Regularized)
Indicator (Neighborhood)	36580.50	35986.5
Linear (GrLivArea)	37797.81	36073.8
Polynomial (GrLivArea ³)	38185.37	36021.6
Piecewise (GrLivArea)	38212.32	36069.4
Age and Renovation	44934.58	N/A (Regularized)
Feature Model	46337.88	36591.4
Dichotomous (HasGarage)	55473.82	37007.8
Log Enhanced Size Quality Model	180909.17	-735.3
Log Feature Model	180909.17	827.5

Best model for predictive power (Lowest MAE): Enhanced Size Quality Model

```
[37]: from scipy.stats import zscore, pearsonr

# Feature creation
# Select relevant features
selected_data = dataframe[["OverallQual", "GrLivArea", "TotalBsmtSF",
    ↪ "SalePrice"]]
extracted_data = selected_data.copy()

# Create a toy feature: predicted price as product of main features
extracted_data["predicted_price"] = (
    extracted_data["OverallQual"] *
    extracted_data["GrLivArea"] *
    extracted_data["TotalBsmtSF"]
)

# Create a relative efficiency feature: how close actual price is to predicted
extracted_data["efficiency"] = extracted_data["SalePrice"] /
    ↪ extracted_data["predicted_price"]

# Display sample
print(extracted_data.head())

# Correlation value
corr, p_value = pearsonr(extracted_data["predicted_price"],
    ↪ extracted_data["SalePrice"])
print("Pearson correlation of Predicted Price to Actual Sale Price:", corr,
    ↪ "p-value:", p_value)
```

	OverallQual	GrLivArea	TotalBsmtSF	SalePrice	predicted_price	efficiency
0	7	1710	856	208500	10246320	0.020349
1	6	1262	1262	181500	9555864	0.018994
2	7	1786	920	223500	11501840	0.019432
3	7	1717	756	140000	9086364	0.015408

4 8 2198 1145 250000 20133680 0.012417
Pearson correlation of Predicted Price to Actual Sale Price: 0.6366702124632712
p-value: 8.969506468597593e-167

```
[ ]: df_test = pd.read_csv('/kaggle/input/
    ↪house-prices-advanced-regression-techniques/test.csv')

# Fill empty values with the training median so predict() doesn't return NaN.
for col in ['OverallQual', 'GrLivArea', 'TotalBsmtSF']:
    df_test[col] = df_test[col].fillna(dataframe[col].median())

# Predict with the fitted multiple model
df_test['SalePrice'] = multiple_model.predict(df_test)

assert df_test['SalePrice'].notnull().all(), "NaNs still present in SalePrice!"

output = df_test[['Id', 'SalePrice']]
output.to_csv('/kaggle/working/predicted_prices.csv', index=False)
print(output.head())
```

```
[64]: # A. Fill missing values using the median from your training data (df_ols)
fill_cols = ['OverallQual', 'GrLivArea', 'TotalBsmtSF', 'GarageCars']
for col in fill_cols:
    df_test[col] = df_test[col].fillna(df_ols[col].median())

# FIX: Force fit the model object if it isn't fitted already
if hasattr(size_qual_model, 'fit') and not hasattr(size_qual_model, 'params'):
    fitted_size_qual_model = size_qual_model.fit()
else:
    fitted_size_qual_model = size_qual_model

# B. Predict using the FITTED results model wrapper
df_test['SalePrice'] = fitted_size_qual_model.predict(df_test)

# Verify no NaNs are present
assert df_test['SalePrice'].notnull().all(), "NaNs still present in SalePrice!"

# C. Export the submission file
output_size_qual = df_test[['Id', 'SalePrice']]
output_size_qual.to_csv('/kaggle/working/submission_size_qual_model.csv',
    ↪index=False)
print("\n--- Size & Quality Model Submission Sample ---")
print(output_size_qual.head())
```

```
--- Size & Quality Model Submission Sample ---
   Id      SalePrice
0 1461  106773.736093
```

```

1 1462 164646.047580
2 1463 160308.310965
3 1464 182674.427111
4 1465 226542.716428

```

```

[65]: import numpy as np
import pandas as pd

# Load the test set
df_test = pd.read_csv('/kaggle/input/
    ↪house-prices-advanced-regression-techniques/test.csv')

# A. Fill missing values using the median from your training data (df_ols)
fill_cols = ['OverallQual', 'GrLivArea', 'TotalBsmtSF', 'GarageCars',
    ↪'YearBuilt']
for col in fill_cols:
    df_test[col] = df_test[col].fillna(df_ols[col].median())

# B. Recreate the exact engineered features used during training
df_test['TotalRealEstateSF'] = df_test['GrLivArea'] + df_test['TotalBsmtSF']
df_test['Qual_x_Area'] = df_test['OverallQual'] * df_test['GrLivArea']

# FIX: Force fit the enhanced model object if it isn't fitted already
if hasattr(log_enhanced_model, 'fit') and not hasattr(log_enhanced_model,
    ↪'params'):
    fitted_log_enhanced = log_enhanced_model.fit()
else:
    fitted_log_enhanced = log_enhanced_model

# C. Predict with the FITTED enhanced log model
log_predictions = fitted_log_enhanced.predict(df_test)

# D. Convert log predictions back to actual dollar values
df_test['SalePrice'] = np.expml(log_predictions)

# Verify no NaNs are present
assert df_test['SalePrice'].notnull().all(), "NaNs still present in SalePrice!"

# E. Export the submission file
output_enhanced = df_test[['Id', 'SalePrice']]
output_enhanced.to_csv('/kaggle/working/submission_enhanced_model.csv',
    ↪index=False)
print("--- Enhanced Model Submission Sample ---")
print(output_enhanced.head())

```

```

--- Enhanced Model Submission Sample ---
   Id      SalePrice
0 1461  115950.007165

```

1	1462	141031.329748
2	1463	178329.211479
3	1464	186085.104979
4	1465	186237.607635

[]: